Ansible and configuration management

Describe servers in inventories and playbooks, keep them consistent with idempotent roles, protect secrets with Vault and roll changes out safely.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

Contents

1. Ansible basics, inventory and ad-hoc commands (14 min)
2. Playbooks, tasks and idempotence (15 min)
3. Variables, facts and templates (15 min)
4. Handlers, conditionals and loops (14 min)
5. Roles, collections and reusable automation (15 min)
6. Ansible Vault, secrets and safe practice (14 min)
7. Ansible at scale, testing and working with Terraform (16 min)
8. Dynamic inventory for cloud environments (11 min)
9. Testing and linting Ansible code (12 min)
10. Debugging Ansible runs, connections and variables (15 min)
11. Ansible performance and execution strategies (12 min)
12. Writing custom Ansible modules and filters (12 min)
Cheat sheet

Lesson 1. Ansible basics, inventory and ad-hoc commands

14 minutes. Understand agentless configuration management, describe your servers in an inventory, and run one-off commands and modules across many hosts.

You have twelve servers that should all have the same web server, the same users and the same firewall rules. Logging in to each one works for a week. After that, one server has a different package version, one has a forgotten manual change, and nobody can say which. Configuration management keeps servers matching a description instead of matching someone's memory.

What you will learn

- What configuration management solves, and where Ansible fits
- How Ansible reaches hosts without installing an agent
- Writing an inventory with groups and variables
- Running ad-hoc commands and modules

How Ansible works

Ansible runs on a **control node** (your laptop or a build server). It connects to **managed nodes** over SSH (or WinRM for Windows), copies a small module to each, runs it, reads the result and removes it. Nothing needs to be installed on the managed nodes except Python and an SSH login.

- **Agentless**: no daemon to patch or secure on each server
- **Declarative modules**: you say "package nginx is present", not "run apt install"
- **Push model**: you decide when changes happen, which suits controlled rollouts

Term	Meaning
Control node	The machine that runs `ansible` commands
Managed node	A server Ansible configures
Inventory	The list of hosts and groups
Module	A unit of work such as `apt`, `copy` or `service`
Playbook	A YAML file that lists tasks to run (next lesson)

The inventory

An inventory names your hosts and sorts them into groups. The INI form is the quickest to read.

```
[web]
web1.example.com
web2.example.com ansible_user=deploy

[db]
db1.example.com

[prod:children]
web
db

[prod:vars]
ansible_python_interpreter=/usr/bin/python3
```

- `[web]` and `[db]` are groups of hosts
- `[prod:children]` makes `prod` a group made of other groups
- `[prod:vars]` sets variables for every host in `prod`

Check what Ansible sees before you change anything:

MARGA by EknathaLabs marga.eknathalabs.com

```
ansible-inventory -i inventory.ini --graph
ansible-inventory -i inventory.ini --host web2.example.com
```

Cloud environments change constantly, so teams often use a **dynamic inventory** plugin that asks the cloud API for current instances instead of a hand-written file.

Ad-hoc commands

An ad-hoc command runs one module on a set of hosts. It is ideal for quick facts and one-off fixes.

```
ansible all -i inventory.ini -m ping
ansible web -i inventory.ini -m command -a "uptime"
ansible web -i inventory.ini -m apt -a "name=nginx state=present" --become
ansible db -i inventory.ini -m service -a "name=postgresql state=started" --become
```

- `-m` chooses the module and `-a` gives its arguments
- `--become` runs with elevated rights (sudo)
- `--check` shows what would change without changing it
- `--limit web1.example.com` restricts the run to part of the inventory

The `ping` module is not an ICMP ping. It checks that SSH works and Python is usable on the target.

Why modules beat shell commands

`state=present` can be run ten times and install the package once. The module reports `changed` only when it altered something. A raw `command: apt install nginx` has no such awareness. Use a module when one exists, and reserve `command` or `shell` for the cases that have none.

Diagram: What happens when you run an Ansible command

Part	What it does	What goes wrong
Control node	Reads the inventory and builds a connection per host.	If the control node is lost, so is your only copy of the automation unless it is stored in version control.
SSH connection	Opens an SSH session to each managed node.	Wrong keys, firewalls or host key prompts stop the run for that host.
Module copied and run	A small Python module runs on the target and does one job.	Missing Python on the target gives a module failure, not a config error.
JSON result	The module returns ok, changed or failed, and is then removed.	Ignoring "failed" on one host can leave the fleet inconsistent.

Flow: Control node to SSH connection / SSH connection to Module copied and run / Module copied and run to JSON result

Incident: the one server nobody could rebuild

> **Incident:** *A payment web server died after a disk fault, and no one could recreate it because years of manual changes lived only on that machine.*

```
$ ssh web3 'dpkg -l | wc -l'
611
$ ssh web1 'dpkg -l | wc -l'
498
```

The team spent two days comparing package lists. Afterwards they described the web tier in an inventory and a playbook, ran it against all servers, and made "no manual changes on servers" a rule. The next failure took twenty minutes to replace.

Common mistakes

- Using `shell` for everything, which gives up change reporting
- Forgetting `--become` and then misreading permission errors
- Keeping the inventory only on one laptop

MARGA by EknathaLabs marga.eknathalabs.com

- Treating `ping` as a network test

Check yourself

What does it mean that Ansible is agentless?

Answer: Nothing needs to be installed on managed nodes beyond SSH access and Python

In an inventory, what does [prod:children] define?

Answer: A group made of other groups

Which option previews changes without making them?

Answer: --check

What does the ping module test?

Answer: That SSH works and Python is usable on the target

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 2. Playbooks, tasks and idempotence

15 minutes. Write YAML playbooks with plays and tasks, run them safely with check and diff modes, and understand why idempotence matters.

Ad-hoc commands fix one thing once. A playbook records the whole intended state of a server in a file you can review, run again and keep in version control. It is the central artifact of Ansible.

What you will learn

- The structure of a playbook: plays, hosts, tasks
- Using common modules for packages, files, services and users
- Idempotence and how to read `changed` and `ok`
- Running with `--check`, `--diff` and `--limit`

A first playbook

```
- name: Configure web servers
  hosts: web
  become: true
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true

    - name: Create the site directory
      ansible.builtin.file:
        path: /var/www/shop
        state: directory
        owner: www-data
        mode: "0755"

    - name: Ensure nginx is running and starts on boot
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

- A **play** maps a group of hosts to tasks
- A **task** calls one module with arguments
- Tasks run top to bottom on each host, and a failed task stops that host
- `name` is optional but it is what you read in the output, so write good ones

Run it:

```
ansible-playbook -i inventory.ini site.yml
ansible-playbook -i inventory.ini site.yml --check --diff
ansible-playbook -i inventory.ini site.yml --limit web1.example.com
ansible-playbook site.yml --syntax-check
```

Idempotence

A task is **idempotent** when running it again changes nothing once the desired state exists. The recap printed at the end of each run tells you:

```
PLAY RECAP
web1 : ok=4  changed=0  unreachable=0  failed=0  skipped=0
```

A healthy second run shows `changed=0`. If it keeps reporting changes, something in your playbook is not describing a state, and it will hide real changes later.

Common causes of fake changes are `command` or `shell` tasks, which always report `changed`. Fix them in one of three ways:

MARGA by EknathaLabs
marga.eknathalabs.com

```
- name: Initialise the application once
  ansible.builtin.command: /opt/shop/bin/init-db
  args:
    creates: /opt/shop/.initialised

- name: Read the version without claiming a change
  ansible.builtin.command: /opt/shop/bin/version
  register: shop_version
  changed_when: false
```

`creates` skips the task when a file exists, and `changed_when: false` marks a read-only task as unchanged.

Useful modules

Need	Module
Packages	`ansible.builtin.apt`, `dnf`, `package`
Files and directories	`file`, `copy`, `template`, `lineinfile`
Services	`service`, `systemd`
Users and keys	`user`, `authorized_key`
Run something	`command`, `shell` (last resort)
Download	`get_url`, `unarchive`

Use the full name such as `ansible.builtin.apt`. It avoids ambiguity when collections from other sources define modules with the same short name.

Check mode and diff

`--check` runs the play without changing anything, and `--diff` prints what would change in files. Together they act as the `plan` step of Ansible. Not every module can predict its effect, so a clean check is good evidence but not a guarantee.

Diagram: Anatomy of a playbook run

Part	What it does	What goes wrong
Play	Picks the hosts and sets options such as become.	A wrong hosts pattern runs on too many or too few servers.
Gather facts	Collects OS and hardware details from each host.	Slow on large fleets, and fails if Python is missing.
Tasks in order	Each task calls a module and reports ok, changed or failed.	A failed task stops the host, leaving it half configured.
Recap	Counts of ok, changed, failed and unreachable per host.	Unread recaps hide hosts that silently failed.

Flow: Play to Gather facts / Gather facts to Tasks in order / Tasks in order to Recap

Incident: the playbook that restarted production every run

Incident: *A team scheduled a playbook every hour. Because one task always reported changed, a handler restarted the application hourly and caused short outages.*

```
$ ansible-playbook site.yml | tail -3
web1 : ok=9  changed=1  unreachable=0  failed=0  skipped=0
web2 : ok=9  changed=1  unreachable=0  failed=0  skipped=0
```

A `shell` task that wrote a timestamp into a config file was reporting a change every time. They replaced it with a `template` task that renders the same content when nothing changes, and added a pipeline check that fails when a second run is not clean.

Common mistakes

- Using `shell` where a module exists
- Skipping the second run that proves idempotence

MARGA by EknathaLabs marga.eknathalabs.com

- Leaving tasks unnamed so output is unreadable
- Running a whole play when `--limit` would test it on one host first

Check yourself

What does a healthy second run of an idempotent playbook show?

Answer: changed=0 for every host

Which option shows what a playbook would change in files without changing them?

Answer: --check --diff

How do you make a read-only command task report no change?

Answer: Add changed_when: false

Why use ansible.builtin.apt instead of apt?

Answer: The full name avoids ambiguity with modules from other collections

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 3. Variables, facts and templates

15 minutes. Parameterise playbooks with variables, understand precedence, use gathered facts and render configuration files with Jinja2 templates.

One playbook should configure staging and production, differing only in values such as ports, hostnames and sizes. Variables and templates let you keep the logic once and the differences in data.

What you will learn

- Defining variables in playbooks, inventory and files
- Variable precedence, and why it surprises people
- Facts gathered from hosts
- Jinja2 templates for configuration files

Where variables live

```
- name: Configure the app
  hosts: web
  become: true
  vars:
    app_port: 8080
  vars_files:
    - vars/common.yml
  tasks:
    - name: Show the port
      ansible.builtin.debug:
        msg: "Listening on {{ app_port }}"
```

Common places, from broad to specific:

- `group_vars/web.yml` applies to every host in the `web` group
- `host_vars/web1.example.com.yml` applies to one host
- `vars` in a play or `vars_files`
- `-e "app_port=9090"` on the command line, which wins over almost everything

Directory layout is a good habit:

```
inventory/
  production/
    hosts.ini
    group_vars/
      all.yml
      web.yml
  staging/
    hosts.ini
    group_vars/
      all.yml
```

Each environment owns its values, and the same playbook runs against either inventory.

Precedence

Ansible has more than twenty precedence levels. You only need the idea: **more specific beats more general, and extra vars (`-e`) beat everything**. To avoid confusion, define each variable in one place, give role defaults a clear name prefix, and use `ansible-inventory --host NAME` to see what a host really receives.

Facts

Before tasks run, Ansible gathers **facts**, details about the host stored in `ansible_facts`.

```
- name: Use facts
  hosts: all
  tasks:
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
    - name: Report the distribution
      ansible.builtin.debug:
        msg: "{{ ansible_facts['distribution'] }} {{ ansible_facts['distribution_version'] }}, {{  \
    ansible_facts['memtotal_mb'] }} MB"
```

```
ansible web1.example.com -i inventory.ini -m setup -a "filter=ansible_distribution*"
```

Facts let a playbook adapt to the operating system or memory size. Gathering costs time on large fleets, so set `gather_facts: false` where you do not use them.

Templates

A template is a text file with Jinja2 placeholders. The `template` module renders it on the control node and copies the result.

```
# nginx site for {{ site_name }}  -- managed by Ansible, do not edit by hand
server {
    listen {{ app_port }};
    server_name {{ site_name }};
{% for upstream in app_backends %}
    # backend {{ upstream }}
{% endfor %}
}
```

```
- name: Render the site configuration
  ansible.builtin.template:
    src: site.conf.j2
    dest: /etc/nginx/conf.d/shop.conf
    owner: root
    group: root
    mode: "0644"
    validate: "nginx -t -c /etc/nginx/nginx.conf"
```

- `{{ }}` prints a value, `{% %}` holds logic such as loops and conditions
- The header comment tells future readers not to edit by hand
- `validate` runs a check on the rendered file before it replaces the old one (some tools need `%s` in the command for the file path)

Filters transform values: `{{ name | upper }}`, `{{ port | default(80) }}`, `{{ items | join(',') }}`. The `default` filter prevents failures when a variable is missing.

Diagram: How a templated file reaches a server

Part	What it does	What goes wrong
Variables and facts	Values from inventory, group_vars, play vars and gathered facts.	A value defined in two places can silently win from the wrong one.
Jinja2 template	A text file with placeholders and simple logic.	Missing variables stop the play or render empty text.
Rendered on control node	The template module fills in the values.	A logic mistake only shows after rendering.
File on the host	Copied only if content differs, then validated.	An unvalidated bad file can break the service on reload.

Flow: Variables and facts to Rendered on control node / Jinja2 template to Rendered on control node / Rendered on control node to File on the host

Incident: the production config with staging values

Incident: *A deployment pointed production at the staging database because a variable defined in two places let the wrong value win.*

```
$ ansible-inventory -i inventory/production/hosts.ini --host web1 | grep db_host
    "db_host": "db.staging.internal",
```

An old `-e @staging.json` was still in a shell alias. The team removed extra-vars files from routine use, put the database host only in `group_vars/production`, and added a check that prints the key variables for

confirmation before the real run.

Common mistakes

- Defining the same variable in many places
- Skipping `default` filters and failing on a missing value
- Editing the deployed file by hand and losing the change at the next run
- Putting secrets in plain variable files (see the vault lesson)

Check yourself

Which source overrides all inventory and play variables?

Answer: Extra vars passed with -e

What is the purpose of gathered facts?

Answer: To give playbooks information about each host, such as OS and memory

What does {% for x in list %} do in a template?

Answer: Runs template logic that repeats a block for each item

Which filter avoids a failure when a variable has not been set?

Answer: default

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 4. Handlers, conditionals and loops

14 minutes. Control task flow with when conditions, loops, handlers that fire on change, block/rescue error handling and tags.

Real servers differ and real changes need follow-up: reload the web server only if its config changed, install a package only on Debian family hosts, create five users without five copied tasks. These tools give a playbook that flow without turning it into a script.

What you will learn

- `when` conditions
- `loop` over lists and dictionaries
- Handlers and when they run
- `block`, `rescue` and `always`
- Tags and limiting a run

Conditions

```
- name: Install on Debian family
  ansible.builtin.apt:
    name: nginx
    state: present
  when: ansible_facts['os_family'] == "Debian"

- name: Install on Red Hat family
  ansible.builtin.dnf:
    name: nginx
    state: present
  when: ansible_facts['os_family'] == "RedHat"
```

`when` takes a plain expression with no braces. Combine conditions with `and`, `or` and `not`, and use `is defined` to test a variable.

Loops

```
- name: Create service users
  ansible.builtin.user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
    state: present
  loop:
    - { name: alice, groups: "sudo" }
    - { name: bob, groups: "developers" }
```

Use `loop_control` with `label` to keep output short when items are large. Do not loop over a package list with many tasks, because most package modules accept a list in `name` and do it in one transaction.

Handlers

A **handler** is a task that runs only when notified, and only once at the end of the play, however many tasks notified it.

```
- name: Web configuration
  hosts: web
  become: true
  tasks:
    - name: Render nginx configuration
      ansible.builtin.template:
        src: shop.conf.j2
        dest: /etc/nginx/conf.d/shop.conf
        validate: "nginx -t -c /etc/nginx/nginx.conf"
      notify: Reload nginx

    - name: Ensure nginx is running
      ansible.builtin.service:
        name: nginx
```

MARGA by EknathaLabs
marga.eknathalabs.com

```
        state: started

  handlers:
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

The reload happens only if the template task reported `changed`. If a later task fails before the handlers run, the handler is skipped unless you use `meta: flush_handlers` or `--force-handlers`, which is a common source of "config changed but service not reloaded".

Handling errors

```
- name: Deploy with a safety net
  block:
    - name: Update the application
      ansible.builtin.copy:
        src: app-v2.tar.gz
        dest: /opt/shop/app.tar.gz
    - name: Restart the application
      ansible.builtin.service:
        name: shop
        state: restarted
  rescue:
    - name: Roll back to the previous release
      ansible.builtin.command: /opt/shop/bin/rollback
  always:
    - name: Report the outcome
      ansible.builtin.debug:
        msg: "Deployment attempt finished"
```

`rescue` runs if something in the block fails, and `always` runs regardless. `ignore_errors: true` is different: it keeps going after a failure and hides the problem, so use it rarely and with a comment.

Tags and limiting

```
ansible-playbook site.yml --tags config
ansible-playbook site.yml --skip-tags slow
ansible-playbook site.yml --list-tasks
```

Tags let you run part of a playbook. Combine with `--limit` to test on one host before the fleet.

Diagram: How a change triggers a reload

Part	What it does	What goes wrong
Config task	Renders the file and reports changed or ok.	An always-changed task triggers needless reloads.
Notify	Queues the handler only if the task changed.	A typo in the handler name means it never runs.
End of play	Queued handlers run once each after all tasks.	A failure before this point may skip the handler.
Handler runs	Reloads the service with the new configuration.	A reload without validation can take the service down.

Flow: Config task to Notify / Notify to End of play / End of play to Handler runs

Incident: the handler that never ran

Incident: *A new certificate was deployed and the play reported success, but the site kept serving the old one for days because the play failed after the copy and before the handlers.*

```
$ ansible-playbook site.yml
TASK [Copy certificate] changed
TASK [Run database check] fatal: FAILED
RUNNING HANDLER [Reload nginx] skipped
```

MARGA by EknathaLabs marga.eknathalabs.com

The team added `meta: flush_handlers` right after the copy task and turned on `--force-handlers` in the pipeline. They also added a post-deploy check that reads the certificate expiry date from the live site.

Common mistakes

- Braces inside `when`
- Using `ignore_errors` instead of fixing or rescuing
- Restarting a service from a normal task (it restarts on every run)
- Putting everything in one giant play with no tags

Check yourself

When does a handler run?

Answer: Once at the end of the play, if a task notified it by reporting changed

What does a rescue section do?

Answer: Runs when a task in the block fails

Which is the correct condition syntax?

Answer: when: ansible_facts['os_family'] == "Debian"

Why is ignore_errors usually a smell?

Answer: It hides failures so later tasks run on a broken base

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 5. Roles, collections and reusable automation

15 minutes. Organise automation into roles with a standard layout, install shared collections and requirements, and keep roles small and testable.

A single playbook grows until nobody wants to touch it. Roles package the tasks, templates, defaults and handlers for one job (for example "nginx" or "postgres") so that many playbooks can reuse them.

What you will learn

- The standard role directory layout
- Role defaults versus vars
- Using roles in a play, and passing variables
- Collections and `requirements.yml`

Role layout

```
roles/
  nginx/
    defaults/main.yml     # lowest-precedence defaults, meant to be overridden
    vars/main.yml         # high-precedence internal constants
    tasks/main.yml        # the work
    handlers/main.yml     # reload and restart handlers
    templates/site.conf.j2
    files/
    meta/main.yml         # dependencies and metadata
```

Create the skeleton with `ansible-galaxy init roles/nginx`. Only `tasks/main.yml` is required.

A small role

```
# roles/nginx/defaults/main.yml
nginx_port: 80
nginx_site_name: example.com

# roles/nginx/tasks/main.yml
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present

- name: Render site configuration
  ansible.builtin.template:
    src: site.conf.j2
    dest: /etc/nginx/conf.d/site.conf
  notify: Reload nginx

# roles/nginx/handlers/main.yml
- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

Use it from a playbook:

```
- name: Web tier
  hosts: web
  become: true
  roles:
    - role: nginx
      vars:
        nginx_port: 8080
        nginx_site_name: shop.example.com
```

Design rules for roles

- **One job per role**, with a clear name and a short README

MARGA by EknathaLabs marga.eknathalabs.com

- **Prefix every variable** with the role name (`nginx_port`) so roles do not collide
- Put overridable settings in `defaults`, and keep `vars` for values users should not change
- Avoid hidden dependencies on other roles' variables
- Keep roles free of environment-specific values; those belong in inventory

Collections

A **collection** bundles modules, plugins and roles for a product or cloud, such as `amazon.aws` or `community.postgresql`. Install the ones you need and pin versions.

```
# requirements.yml
collections:
  - name: amazon.aws
    version: 8.2.1
  - name: community.general
    version: 9.5.0
roles:
  - name: geerlingguy.nginx
    version: 3.2.0

ansible-galaxy install -r requirements.yml
ansible-galaxy collection list
```

Pinning matters. An unpinned role or collection can change under you and make last month's playbook behave differently. Review third-party roles before using them in production, because they run with your privileges.

Diagram: From playbook to configured server using roles

Part	What it does	What goes wrong
Playbook	Lists hosts and the roles to apply.	A play with no pinned role versions can change between runs.
Role	Tasks, templates, handlers and defaults for one job.	Variables without a prefix collide with other roles.
Collection	Supplies modules and plugins that the role calls.	An unpinned collection can change behaviour after an update.
Managed node	Receives the rendered files and service changes.	A role that assumes one OS breaks on another.

Flow: Playbook to Role / Role to Collection / Role to Managed node

Incident: the role update that broke a deploy

> **Incident:** *An unpinned third-party role was updated upstream and changed a default port. The next scheduled run reconfigured production web servers on the wrong port.*

```
$ ansible-galaxy role list
- geerlingguy.nginx, (unknown version)
```

After the outage the team pinned every role and collection in `requirements.yml`, mirrored the ones they depend on internally, and updated them on purpose in a pull request that runs the tests.

Common mistakes

- Giant roles that install five unrelated things
- Unprefixed variables
- Using unpinned roles from the internet
- Putting production hostnames inside roles

Check yourself

Where should an overridable setting such as a port live in a role?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: defaults/main.yml

What is a collection?

Answer: A distributable bundle of modules, plugins and roles

Why prefix role variables with the role name?

Answer: To avoid collisions between roles that share the same play

Why pin versions in requirements.yml?

Answer: So runs are repeatable and upstream changes arrive only when you choose

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 6. Ansible Vault, secrets and safe practice

14 minutes. Keep passwords and keys out of plain text with Ansible Vault, hide secrets from logs, limit privilege escalation and review what automation can reach.

Automation holds the keys to your servers: SSH access, sudo rights, database passwords and API tokens. If those leak, the blast radius is every machine the automation can reach, so secret handling is part of the job, not an extra.

What you will learn

- Encrypting values and files with Ansible Vault
- Keeping secrets out of logs with `no_log`
- Using an external secret manager instead
- Least privilege for the automation itself

Ansible Vault

Vault encrypts a file or a single value so it can sit next to your playbooks. Without the vault password it is unreadable.

```
ansible-vault create group_vars/prod/vault.yml
ansible-vault edit group_vars/prod/vault.yml
ansible-vault encrypt_string 's3cret-value' --name 'vault_db_password'
ansible-playbook site.yml --vault-id prod@prompt
```

A common pattern keeps readable variable names pointing to encrypted ones:

```
# group_vars/prod/vars.yml (readable, safe to review)
db_password: "{{ vault_db_password }}"

# group_vars/prod/vault.yml (encrypted with ansible-vault)
vault_db_password: placeholder-replaced-by-encrypted-content
```

Reviewers can see which variables exist and where they are used, while the values stay encrypted. Use separate vault IDs for different environments so a developer with staging access cannot read production secrets.

Where the vault password comes from

- Prompt interactively (`--vault-id prod@prompt`)
- A password file outside the repository with tight permissions
- A script that fetches it from your secret manager or CI secret store

Never commit the vault password next to the encrypted files. That defeats the point.

Keep secrets out of output

Verbose output and logs can print task arguments. Mark sensitive tasks:

```
- name: Set the database password
  community.postgresql.postgresql_user:
    name: shop
    password: "{{ db_password }}"
  no_log: true
```

`no_log: true` hides the task's arguments and results, including useful error details, so enable it only for tasks that handle secrets and debug with a safe test copy.

MARGA by EknathaLabs
marga.eknathalabs.com

Better: do not store long-lived secrets at all

Vault is fine for small teams. Larger teams often fetch secrets at run time from a manager such as HashiCorp Vault or a cloud secret service using a lookup plugin, so secrets are rotated and audited in one place. Whatever you choose, rotate any secret that has ever appeared in a log or a repository.

Least privilege for the automation

- Use a dedicated SSH key or short-lived certificate for Ansible, not a person's key
- Give the account only the sudo commands it needs where possible
- Limit which hosts the control node can reach with network rules
- Protect the control node like production: patch it, restrict logins, log access
- Review third-party roles and collections before running them as root

Diagram: How a secret travels through a run

Part	What it does	What goes wrong
Encrypted vault file	The value is stored encrypted in the repository.	A leaked vault password exposes every value in the file.
Vault password source	Prompt, protected file or secret manager.	A password committed with the files cancels the encryption.
Playbook run	Decrypts in memory and renders tasks.	Verbose output may print values without no_log.
Target host	The secret is written to a protected file or passed to a service.	A world-readable file undoes the protection.

Flow: Encrypted vault file to Playbook run / Vault password source to Playbook run / Playbook run to Target host

Incident: the password in the build log

Incident: *A database password appeared in a CI build log because a task ran with verbose output and no* `no_log`*. The log was readable by the whole company.*

```
$ ansible-playbook site.yml -vvv | grep -i password
"password": "REDACTED-IN-THIS-EXAMPLE"
```

The team rotated the password, added `no_log` to every task that handles credentials, restricted who can read pipeline logs, and added a scanner that fails the pipeline if secrets appear in output.

Common mistakes

- Committing the vault password file
- One vault password for all environments
- Using `no_log` everywhere so failures become undebuggable
- Running automation with a personal, over-privileged account

Check yourself

What does ansible-vault do?

Answer: Encrypts files or values so they can be stored with playbooks

Which setting hides a task's arguments and results from output?

Answer: no_log: true

Why use separate vault IDs per environment?

Answer: People with access to one environment cannot decrypt another environment's secrets

Where should the vault password be kept?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: Outside the repository, entered at run time or fetched from a secret manager

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 7. Ansible at scale, testing and working with Terraform

16 minutes. Roll out changes safely with serial batches and limits, test roles with linting and Molecule, scale connections, and divide the work between Terraform and Ansible.

Running a playbook against three servers is easy. Running it against three hundred during business hours means batches, health checks, a way to stop, and tests that run before the change reaches anyone.

What you will learn

- Rolling updates with `serial` and failure limits
- Linting and testing roles
- Performance settings for large inventories
- Where Ansible stops and Terraform starts
- A controller such as AWX or a pipeline

Rolling updates

```
- name: Rolling update of the web tier
  hosts: web
  become: true
  serial: "20%"
  max_fail_percentage: 10
  tasks:
    - name: Take the host out of the load balancer
      ansible.builtin.command: /opt/tools/lb-drain {{ inventory_hostname }}
      delegate_to: localhost
    - name: Update the application
      ansible.builtin.apt:
        name: shop-app
        state: latest
    - name: Wait for the health endpoint
      ansible.builtin.uri:
        url: "http://localhost:8080/health"
        status_code: 200
      register: health
      retries: 10
      delay: 5
      until: health.status == 200
    - name: Put the host back
      ansible.builtin.command: /opt/tools/lb-enable {{ inventory_hostname }}
      delegate_to: localhost
```

- `serial` updates a batch at a time instead of everything at once
- `max_fail_percentage` aborts the play when too many hosts fail
- `retries` with `until` waits for readiness rather than guessing a sleep
- `delegate_to` runs a task on another machine such as a load balancer or localhost
- `any_errors_fatal: true` stops the whole play at the first failure when a partial rollout is worse

Test before you roll

```
ansible-lint site.yml
yamllint .
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --check --diff --limit web1.example.com
```

For roles, **Molecule** creates a throwaway container or VM, applies the role, runs it a second time to prove idempotence, then runs verification checks. A pipeline that runs lint, Molecule and a staging run catches most mistakes before production.

MARGA by EknathaLabs marga.eknathalabs.com

Speed

- Turn off `gather_facts` or use `gather_subset` when you do not need all facts
- Raise `forks` (parallel connections, default 5) to match your control node and network
- Enable SSH connection reuse (`ControlPersist`) and pipelining
- Use `strategy: free` only when hosts are independent
- Avoid loops that call a module once per item when it accepts a list

Ansible and Terraform

They solve neighbouring problems and work well together.

Question	Better fit
Create networks, servers, databases, load balancers	Terraform
Install packages, render configuration, manage users on a server	Ansible
Keep a machine matching a description over time	Ansible, or rebuild from an image
Build the server image once	A tool such as Packer, then Ansible inside it

A common flow: Terraform creates instances and outputs addresses, a dynamic inventory reads them, and Ansible configures them. For container and Kubernetes workloads, immutable images often replace much of host configuration, but Ansible still manages the hosts that run them.

Running it for a team

Running playbooks from laptops makes results depend on who ran them. A shared controller (AWX or the supported Automation Platform) or a CI pipeline gives you credentials kept in one place, job history, role-based access, scheduling and a standard environment.

Diagram: A safe fleet rollout

Part	What it does	What goes wrong
Lint and test	ansible-lint, Molecule and check mode on a test host.	Skipping tests sends mistakes to every server at once.
Canary batch	Update one small batch first and verify health.	A canary without a health check proves nothing.
Serial waves	Continue in batches, draining and restoring each host.	Without a failure limit one bad change spreads fleet-wide.
Verify	Check version, health and error rates after each wave.	No verification means failures are found by users.

Flow: Lint and test to Canary batch / Canary batch to Serial waves / Serial waves to Verify

Incident: the update that went to every server at once

Incident: *A playbook with no `serial` upgraded a library on all web servers simultaneously. The new version had a startup bug, and every server was down at the same moment.*

```
$ ansible-playbook upgrade.yml --list-hosts | tail -1
  hosts (240):
```

The team added `serial` with a canary batch, a health check between waves and `max_fail_percentage`, and made the pipeline refuse to run a production playbook that has none of these.

Common mistakes

- Running all hosts at once during business hours
- Using fixed sleeps instead of health checks

MARGA by EknathaLabs
marga.eknathalabs.com

- Using Ansible to create and destroy cloud infrastructure that Terraform already owns
- Letting laptops run production changes

Check yourself

What does serial do in a play?

Answer: Processes hosts in batches rather than all at once

Which tool is designed to test Ansible roles in disposable instances?

Answer: Molecule

Which pairing fits best?

Answer: Terraform creates the servers and Ansible configures them

Why wait for a health endpoint with retries and until?

Answer: To continue only when the service is actually ready, not after a guessed delay

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 8. Dynamic inventory for cloud environments

11 minutes. Let Ansible discover hosts from the cloud by tags instead of maintaining a hand-written list.

Servers in the cloud appear and disappear all day through autoscaling. A static inventory file is out of date by lunch. A dynamic inventory asks the cloud which machines exist right now and groups them by tag, so playbooks always target the real fleet.

What you will learn

- Inventory plugins and how they query the cloud
- Grouping and variables from tags
- Combining static and dynamic sources
- Checking and troubleshooting the result

Inventory plugins

An inventory plugin reads a small YAML file that tells Ansible where to look.

```
# inventory/aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions:
  - eu-west-1
filters:
  tag:Managed: ansible
  instance-state-name: running
keyed_groups:
  - key: tags.Role
    prefix: role
  - key: tags.Env
    prefix: env
hostnames:
  - private-ip-address
compose:
  ansible_host: private_ip_address
```

The plugin calls the cloud API with your credentials, creates groups such as `role_web` and `env_prod`, and sets variables from instance details. Use roles or instance profiles for credentials, not keys in the file.

Target by tag

Playbooks target the generated groups. Intersection of groups limits the scope:

```
- hosts: role_web:&env_prod
  serial: 2
  roles:
    - nginx
```

The tag is now the contract: if a team forgets the `Role` tag, the host is not configured. Make tags mandatory through your provisioning code. Combine this with `serial` so that a change touches a few hosts at a time.

Mix with static inventory

A directory can contain several sources, and Ansible merges them:

```
inventory/
  aws_ec2.yml
  static.ini
  group_vars/
    all.yml
    role_web.yml
```

Use static files for things the cloud does not know, such as network devices or an office server. Group variables in `group_vars/role_web.yml` apply to the dynamic group automatically.

MARGA by EknathaLabs
marga.eknathalabs.com

Check what it found

```
$ ansible-inventory -i inventory/aws_ec2.yml --graph
$ ansible-inventory -i inventory/aws_ec2.yml --host 10.0.3.21
$ ansible -i inventory/ role_web -m ping
```

If groups are empty, check the filters, region and credentials first. The inventory cache can hide changes: set a short cache timeout, or disable it while testing. Remember that instances that are still booting may not be ready for SSH, so wait for them in the playbook.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Dynamic inventory for cloud environments fits together

Part	What it does	What goes wrong
Inventory plugins	An inventory plugin reads a small YAML file that tells Ansible where to look.	Credentials stored in the inventory file.
Target by tag	Playbooks target the generated groups.	Static host lists for autoscaled fleets.
Mix with static inventory	A directory can contain several sources, and Ansible merges them.	Stale inventory cache.
Check what it found	If groups are empty, check the filters, region and credentials first.	Check the Common mistakes list below this diagram for what usually goes wrong here.

Flow: Inventory plugins to Target by tag / Target by tag to Mix with static inventory / Mix with static inventory to Check what it found

Incident: The server that nobody patched

Incident: *A monthly patch run reported success, but a security scan later found 6 servers still unpatched.*

```
$ ansible-inventory -i hosts.ini --graph | grep -c web
12
$ aws ec2 describe-instances --filters Name=tag:Role,Values=web --query "length(Reservations[].Ins \
    tances[])"
18
```

The hand-written inventory listed 12 web servers while the autoscaler had grown the group to 18. The team moved to a dynamic inventory keyed on tags, required the tags in the provisioning module, and added a pre-run check that compares the inventory count with the cloud count. Patch runs now cover the whole fleet.

Common mistakes

- Static host lists for autoscaled fleets
- Untagged or wrongly tagged hosts
- Stale inventory cache
- Credentials stored in the inventory file

Check yourself

Why use a dynamic inventory in the cloud?

Answer: Hosts change constantly, and the inventory should reflect the real fleet

What creates groups such as role_web?

Answer: keyed_groups built from instance tags

Which command shows the groups Ansible discovered?

Answer: ansible-inventory --graph

MARGA by EknathaLabs marga.eknathalabs.com

What limits how many hosts change at once?

Answer: serial in the play

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 9. Testing and linting Ansible code

12 minutes. Catch mistakes before production with ansible-lint, syntax checks, check mode, Molecule and idempotence tests.

A one-character YAML mistake in a role reached production and restarted every web server at once. Infrastructure code is code, and it deserves the same safety net: lint in the editor, tests in the pipeline, and a rehearsal on disposable machines.

What you will learn

- Static checks: syntax and ansible-lint
- Check and diff mode
- Molecule: test roles on disposable instances
- Idempotence tests and a pipeline layout

Static checks

```
$ ansible-playbook site.yml --syntax-check
$ ansible-lint
```

`ansible-lint` flags risky patterns: shell where a module exists, missing `name`, tasks that are never idempotent, deprecated syntax and unquoted file modes. Keep a `.ansible-lint` file with the rules you follow and fix the warnings rather than ignoring them.

```
# .ansible-lint
profile: production
exclude_paths:
  - molecule/
```

Check and diff mode

```
$ ansible-playbook site.yml --check --diff --limit role_web
```

Check mode predicts changes without making them, and diff mode shows the exact file changes. It is excellent for review, but not perfect: tasks that depend on earlier changes, or modules that cannot predict (for example `command`), may be skipped. Treat it as a preview, not a guarantee.

Molecule

Molecule creates disposable containers or virtual machines, applies a role, verifies the result, and destroys them.

```
# molecule/default/molecule.yml
driver:
  name: docker
platforms:
  - name: debian12
    image: debian:12
    pre_build_image: true
provisioner:
  name: ansible
verifier:
  name: ansible
```

```
# molecule/default/verify.yml
- hosts: all
  tasks:
    - name: Nginx answers
      ansible.builtin.uri:
        url: http://localhost
        status_code: 200
```

`molecule test` runs the sequence. Test the platforms you really use, for example two operating system versions.

MARGA by EknathaLabs
marga.eknathalabs.com

Idempotence and pipeline

A good role changes something the first time and nothing the second. Molecule checks this: the second converge must report `changed=0`. A task that always reports changed (often a raw `command`) hides real changes and makes check mode useless; add `changed_when` or use a module.

A sensible pipeline: lint and syntax check on every commit, Molecule on pull requests, check and diff mode against staging before production, and a canary run with `serial: 1` first.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Testing and linting Ansible code fits together

Part	What it does	What goes wrong
Static checks	ansible-lint flags risky patterns: shell where a module exists, missing name, tasks that are never idempotent, deprecated syntax and unquoted file modes.	Ignoring lint warnings.
Check and diff mode	Check mode predicts changes without making them, and diff mode shows the exact file changes.	Trusting check mode as a guarantee.
Molecule	Molecule creates disposable containers or virtual machines, applies a role, verifies the result, and destroys them.	Testing only on one operating system version.
Idempotence and pipeline	A good role changes something the first time and nothing the second.	Roles that are not idempotent.

Flow: Static checks to Check and diff mode / Check and diff mode to Molecule / Molecule to Idempotence and pipeline

Incident: The task that always changed

> **Incident:** *A nightly Ansible run reported 14 changes every night, so nobody noticed when a real, unwanted change appeared.*

```
PLAY RECAP
web-01 : ok=42 changed=14 unreachable=0 failed=0
$ ansible-lint roles/base
roles/base/tasks/main.yml:31: no-changed-when: Commands should not change things if nothing needs  \
    doing
```

Several tasks used shell commands that always reported changed. The team replaced them with proper modules or added changed_when conditions, then added Molecule idempotence tests to the pipeline. Nightly runs now report zero changes in the normal case, so a non-zero number is a genuine signal worth an alert.

Common mistakes

- Ignoring lint warnings
- Trusting check mode as a guarantee
- Roles that are not idempotent
- Testing only on one operating system version

Check yourself

What does ansible-lint do?

Answer: Flags risky or outdated patterns in playbooks and roles

What does --check --diff show?

Answer: Predicted changes and file differences without applying them

MARGA by EknathaLabs marga.eknathalabs.com

What is Molecule used for?

Answer: Testing roles on disposable instances

What should a second run of a role report?

Answer: changed=0

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 10. Debugging Ansible runs, connections and variables

15 minutes. Find why a playbook failed, a host is unreachable, a variable has the wrong value, or a run is slow, using verbosity, debug, step mode and logs.

Ansible failures usually fall into four groups: the connection did not work, privilege escalation did not work, the data (variables, templates) was wrong, or the module failed on the target. Naming the group tells you where to look.

What you will learn

- Reading the failure message and the recap
- Using `-v`, `debug`, `--step` and `--start-at-task`
- Fixing connection, privilege and Python problems
- Finding the real value of a variable
- Finding slow tasks

Four groups of failure

Group	Typical message	First step
Connection	`UNREACHABLE! ... Permission denied (publickey)`	`ssh -vvv user@host` with the same key
Privilege	`Missing sudo password`, `Permission denied`	Check become, sudoers and the user
Data	an error naming a missing variable, or a template error	debug the variable, `ansible-inventory --host`
Module on target	`FAILED! ... No package matching`	Read the module message, run it ad hoc with `-vvv`

Verbosity and a closer look

```
ansible-playbook site.yml -v          # task results
ansible-playbook site.yml -vvv        # connection details
ansible web1.example.com -i inventory.ini -m setup | head
ansible-playbook site.yml --start-at-task "Render nginx configuration"
ansible-playbook site.yml --step      # confirm each task
ansible-playbook site.yml --check --diff --limit web1.example.com
ansible-config dump --only-changed
```

`-vvv` shows the SSH command, which you can run by hand. `--step` and `--start-at-task` help when a long play fails near the end and you do not want to repeat the first twenty tasks.

Look at the data

```
- name: Show what the host really sees
  ansible.builtin.debug:
    var: db_host

- name: Show a computed value
  ansible.builtin.debug:
    msg: "{{ app_backends | length }} backends, first is {{ app_backends | first }}"
```

```
ansible-inventory -i inventory.ini --host web1.example.com
ansible-inventory -i inventory.ini --list --yaml
```

`ansible-inventory --host` shows variables from inventory sources, which is the quickest way to catch a wrong value from the wrong place.

Typical fixes

- **Permission denied (publickey)**: wrong user or key. Confirm with plain `ssh`, set `ansible_user` and `ansible_ssh_private_key_file`.
- **Host key verification failed**: add the host key properly rather than turning checking off for all hosts.
- **Python not found on target**: install Python, or set `ansible_python_interpreter`.
- **Missing sudo password**: allow passwordless sudo for the automation user, or provide `--ask-become-pass`.
- **Missing variable in a template**: define it, or use `default`, and fail early with an `assert` task that explains what is missing.
- **Slow run**: use the `profile_tasks` callback to list the slowest tasks, then reduce fact gathering or raise forks.

```
- name: Stop early if required data is missing
  ansible.builtin.assert:
    that:
      - db_host is defined
      - app_port | int > 1024
    fail_msg: "db_host must be set and app_port must be above 1024"
```

Do not "fix" a failure by adding `ignore_errors`. That moves the problem somewhere harder to see.

Diagram: Narrowing down a failing Ansible run

Part	What it does	What goes wrong
Read the error	Note the host, task name and message type.	Skipping to retries wastes time and may repeat damage.
Connection and privilege	ssh -vvv, become settings, sudoers.	Wrong user or key looks like a random failure.
Variables and templates	debug, ansible-inventory --host, assert tasks.	A value from the wrong file gives wrong but valid output.
Module on the target	Run the module ad hoc with -vvv on one host.	Missing packages or Python surface only at run time.
Fix and re-run	Change code or data, test on one host, then run the fleet.	Re-running everything without a canary repeats the failure widely.

Flow: Read the error to Connection and privilege / Read the error to Variables and templates / Connection and privilege to Module on the target / Variables and templates to Module on the target / Module on the target to Fix and re-run

Incident: the template that rendered an empty list

Incident: *A firewall template used a variable that was empty on one group of servers. Ansible rendered a valid file with no allow rules, and the load balancer health checks started failing.*

```
$ ansible-inventory -i inventory/production/hosts.ini --host app7 | grep allowed
    "allowed_sources": [],
```

A missing entry in `group_vars` for that group caused it. The team added an `assert` that the list is not empty, a `--check --diff` step in the pipeline, and a canary host before wider rollout.

Common mistakes

- Running with `ignore_errors` to make the red go away
- Debugging on all hosts instead of one with `--limit`
- Turning off host key checking for everyone
- Not reading the line above the error

Check yourself

What does -vvv show that helps with connection failures?

MARGA by EknathaLabs marga.eknathalabs.com

Answer: The SSH command and connection details used

Which command shows the variables a host really receives from inventory?

Answer: ansible-inventory --host NAME

How can a long play be resumed near the failure?

Answer: --start-at-task with the task name

Why is an assert task useful?

Answer: It fails early with a clear message when required data is missing or invalid

MARGA by EknathaLabs
marga.eknathalabs.com

Lesson 11. Ansible performance and execution strategies

12 minutes. Speed up Ansible runs with forks, pipelining, fact caching and smarter strategies, and control rollouts with serial and error limits.

Ansible is agentless, so every task costs an SSH round trip. On a few servers that is invisible. On hundreds it is the whole runtime, and the defaults are conservative.

What you will learn

- Forks, pipelining and persistent connections
- Facts: gathering, caching and skipping
- Strategies, async and batches
- Failure limits for safe rollouts

Connection tuning

```
# ansible.cfg
[defaults]
forks = 50
gathering = smart
fact_caching = jsonfile
fact_caching_connection = /tmp/ansible-facts
fact_caching_timeout = 3600

[ssh_connection]
pipelining = True
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
```

Forks is how many hosts run in parallel. **Pipelining** reduces SSH operations per task, and **ControlPersist** reuses one connection per host. Pipelining needs `requiretty` disabled on targets.

Facts

Gathering facts takes time on every host. Cache them, or turn it off when a play does not use them.

```
- hosts: web
  gather_facts: false
  tasks:
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Strategies and async

The default **linear** strategy waits for each task on all hosts. The **free** strategy lets each host proceed independently, which helps when hosts differ in speed. Use `async` with `poll: 0` for long tasks such as package upgrades, and check later.

Safe rollouts

```
- hosts: web
  serial: [1, 5, "20%"]
  max_fail_percentage: 10
  any_errors_fatal: false
  tasks:
    - name: Deploy release
      ansible.builtin.include_role:
        name: app_release
    - name: Run the database migration once
      ansible.builtin.command: /opt/app/migrate
      run_once: true
```

MARGA by EknathaLabs
marga.eknathalabs.com

`serial` limits how many hosts change at a time, starting with a canary. `max_fail_percentage` stops the play if too many fail. `run_once` runs a task on one host only. Profile slow runs with the `profile_tasks` callback.

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Ansible performance and execution strategies fits together

Part	What it does	What goes wrong
Connection tuning	Forks is how many hosts run in parallel.	Gathering facts that are never used.
Facts	Gathering facts takes time on every host.	Changing all hosts at once.
Strategies and async	The default linear strategy waits for each task on all hosts.	Default forks on a large fleet.
Safe rollouts	serial limits how many hosts change at a time, starting with a canary.	No failure limit during rollouts.

Flow: Connection tuning to Facts / Facts to Strategies and async / Strategies and async to Safe rollouts

Incident: A forty minute run for a reload

Incident: *A simple change took forty minutes because of fact gathering and few forks.*

```
PLAY [web] ***************
Gathering Facts: 31m (forks=5)
Reload nginx: 2m
total time: 40m12s
```

The play gathered full facts it never used and ran only five hosts at a time. Turning off unneeded facts, caching the rest and raising forks reduced the total to eight minutes. A canary batch with serial was added for safety.

Common mistakes

- Default forks on a large fleet
- Gathering facts that are never used
- Changing all hosts at once
- No failure limit during rollouts

Check yourself

What does pipelining do?

Answer: Reduces SSH operations per task

What does serial do?

Answer: Limits how many hosts change at once

What does run_once do?

Answer: Runs a task on a single host

What does max_fail_percentage do?

Answer: Stops the play when too many hosts fail

MARGA by EknathaLabs marga.eknathalabs.com

Lesson 12. Writing custom Ansible modules and filters

12 minutes. Extend Ansible with a small Python module or filter plugin, keep it idempotent, support check mode and test it.

Most tasks have a module already. When yours does not, a small Python module is better than a pile of shell commands, because it can check state, support dry runs and explain what changed.

What you will learn

- Module structure: arguments, state check, change, result
- Idempotence and check mode
- Filter plugins for data transformation
- Testing and distributing with collections

A small module

```
#!/usr/bin/python
from ansible.module_utils.basic import AnsibleModule
import os

def main():
    module = AnsibleModule(
        argument_spec=dict(
            path=dict(type="path", required=True),
            content=dict(type="str", required=True),
        ),
        supports_check_mode=True,
    )
    path, content = module.params["path"], module.params["content"]
    current = open(path).read() if os.path.exists(path) else None
    changed = current != content
    if changed and not module.check_mode:
        with open(path, "w") as f:
            f.write(content)
    module.exit_json(changed=changed, path=path)

if __name__ == "__main__":
    main()
```

The module declares its arguments, reads the **current state**, compares it with the **desired state** and acts only if they differ. With `check_mode`, it reports what would change without doing it. Use `fail_json(msg=...)` for errors.

Filters

```
# plugins/filter/bytes_human.py
class FilterModule:
    def filters(self):
        return {"bytes_human": self.bytes_human}

    def bytes_human(self, n):
        for unit in ["B", "KiB", "MiB", "GiB"]:
            if n < 1024:
                return f"{n:.0f} {unit}"
            n /= 1024
        return f"{n:.1f} TiB"
```

Use it as `{{ ansible_memtotal_mb * 1048576 | bytes_human }}`. Filters are pure functions and should not touch the system.

MARGA by EknathaLabs marga.eknathalabs.com

Test and ship

Unit test the logic with pytest, run the module in a playbook twice and check that the second run reports no change, and run with `--check --diff`. Package modules and plugins in a **collection** with a version and documentation, and keep them in the same repository as the roles that use them.

```
ansible-playbook site.yml --check --diff
ansible-test sanity --docker
```

How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

Diagram: How Writing custom Ansible modules and filters fits together

Part	What it does	What goes wrong
A small module	The module declares its arguments, reads the current state, compares it with the desired state and acts only if they differ.	No check mode support.
Filters	Use it as {{ ansible_memtotal_mb * 1048576 / bytes_human }}.	Check the Common mistakes list below this diagram for what usually goes wrong here.
Test and ship	Unit test the logic with pytest, run the module in a playbook twice and check that the second run reports no change, and run with --check --diff.	Shell commands that always report changed.

Flow: A small module to Filters / Filters to Test and ship

Incident: Always changed

Incident: *A shell-based task reported changed on every run and hid real changes.*

```
TASK [create user] changed: [web-01]
TASK [create user] changed: [web-01]   (second run)
PLAY RECAP: changed=1 on every run
```

A command task created a user and always reported a change, which made review of real changes impossible. The team wrote a module that checked whether the user existed first, supported check mode and was tested for idempotence in CI.

Common mistakes

- Shell commands that always report changed
- No check mode support
- Writing a module when one exists
- No tests for idempotence

Check yourself

What makes a module idempotent?

Answer: It checks current state before changing

What does check mode do?

Answer: Reports changes without making them

What should a filter do?

Answer: Transform data without side effects

How ship custom modules?

Answer: In a versioned collection

MARGA by EknathaLabs marga.eknathalabs.com

Cheat sheet: Ansible

Inventory, ad-hoc commands, playbook structure, variables, handlers, roles, Vault and safe rollout flags.

Ad-hoc commands

```
ansible all -i inventory.ini -m ping
ansible web -i inventory.ini -m command -a "uptime"
ansible web -i inventory.ini -m apt -a "name=nginx state=present" --become
ansible-inventory -i inventory.ini --graph
ansible-inventory -i inventory.ini --host web1.example.com
```

Flag	Meaning
`-m`	Module to run
`-a`	Module arguments
`--become`	Run with elevated rights
`--limit`	Restrict to some hosts
`--check`	Show what would change

Running playbooks

```
ansible-playbook -i inventory.ini site.yml
ansible-playbook site.yml --syntax-check
ansible-playbook site.yml --check --diff --limit web1.example.com
ansible-playbook site.yml --tags config --skip-tags slow
ansible-playbook site.yml --start-at-task "Render nginx configuration"
ansible-playbook site.yml --list-tasks
```

Playbook skeleton

```
- name: Configure web servers
  hosts: web
  become: true
  vars:
    app_port: 8080
  tasks:
    - name: Install nginx
      ansible.builtin.apt:
        name: nginx
        state: present
    - name: Render configuration
      ansible.builtin.template:
        src: shop.conf.j2
        dest: /etc/nginx/conf.d/shop.conf
        validate: "nginx -t -c /etc/nginx/nginx.conf"
      notify: Reload nginx
  handlers:
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Read the recap: a clean second run shows `changed=0`.

Control flow

```
- name: Only on Debian
  ansible.builtin.apt:
    name: nginx
  when: ansible_facts['os_family'] == "Debian"

- name: Make users
  ansible.builtin.user:
    name: "{{ item }}"
  loop: [alice, bob]
```

Tool	Use
when	Run a task only if true

MARGA by EknathaLabs marga.eknathalabs.com

Tool	Use
`loop`	Repeat for each item
`block` / `rescue` / `always`	Try, catch, finally
`changed_when: false`	Mark a read-only task as unchanged
`no_log: true`	Hide secrets from output

Roles and collections

```
ansible-galaxy init roles/nginx
ansible-galaxy install -r requirements.yml
ansible-galaxy collection list
```

Pin versions in `requirements.yml`. Put overridable settings in `defaults/main.yml` and prefix variable names with the role name.

Vault

```
ansible-vault create group_vars/prod/vault.yml
ansible-vault edit group_vars/prod/vault.yml
ansible-vault encrypt_string 'value' --name 'vault_db_password'
ansible-playbook site.yml --vault-id prod@prompt
```

Keep the vault password out of the repository.

Safe rollout

```
- hosts: web
  serial: "20%"
  max_fail_percentage: 10
```

Test with `ansible-lint` and Molecule. Use a canary batch and a health check before continuing.

Troubleshooting

Symptom	First step
UNREACHABLE, publickey	`ssh -vvv` with the same user and key
Missing sudo password	Check become and sudoers
Missing variable	`ansible-inventory --host`, add `default` or `assert`
Slow run	`gather_facts: false`, more forks, `profile_tasks`

Warning: `ansible-playbook` *without* `--limit` *or* `serial` *changes every host in the group at once. Test on one host first.*

Danger: *Never commit the vault password next to the encrypted files. It cancels the protection.*

Warning: `ignore_errors: true` *hides failures and lets later tasks run on a broken base. Fix the cause or use* `rescue`*.*

Warning: `shell` *and* `command` *always report* `changed` *unless you add* `creates` *or* `changed_when`*.*

MARGA by EknathaLabs marga.eknathalabs.com